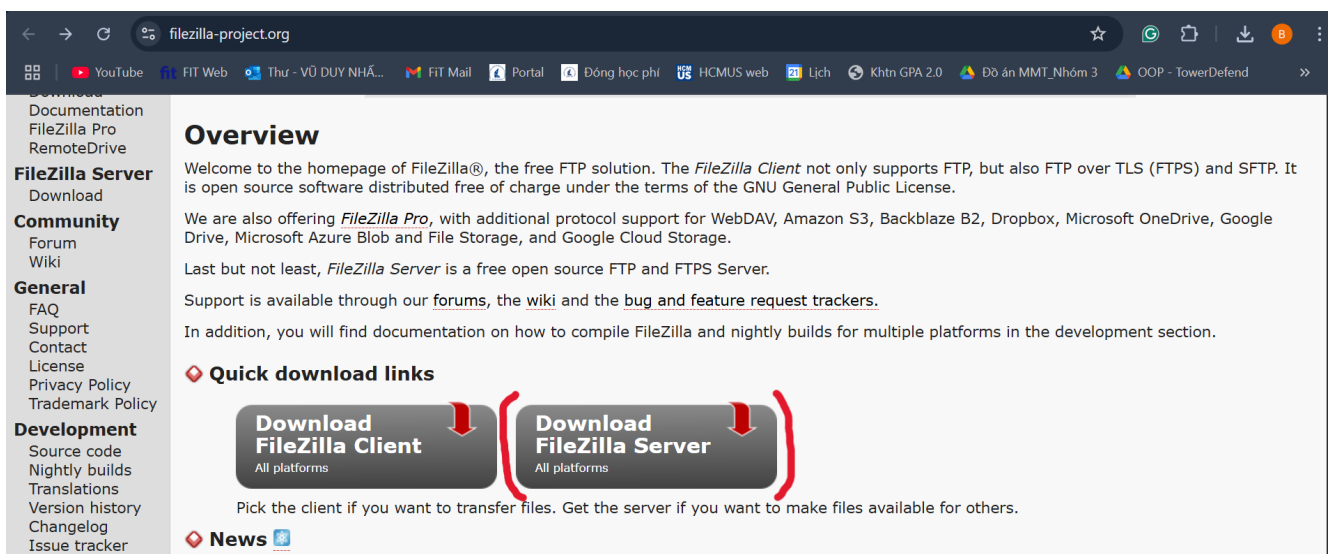


Hướng dẫn cài đặt và sử dụng hệ thống FTP Client - Server an toàn với ClamAV

I. Cài đặt FileZilla Server và các phần mềm cần thiết

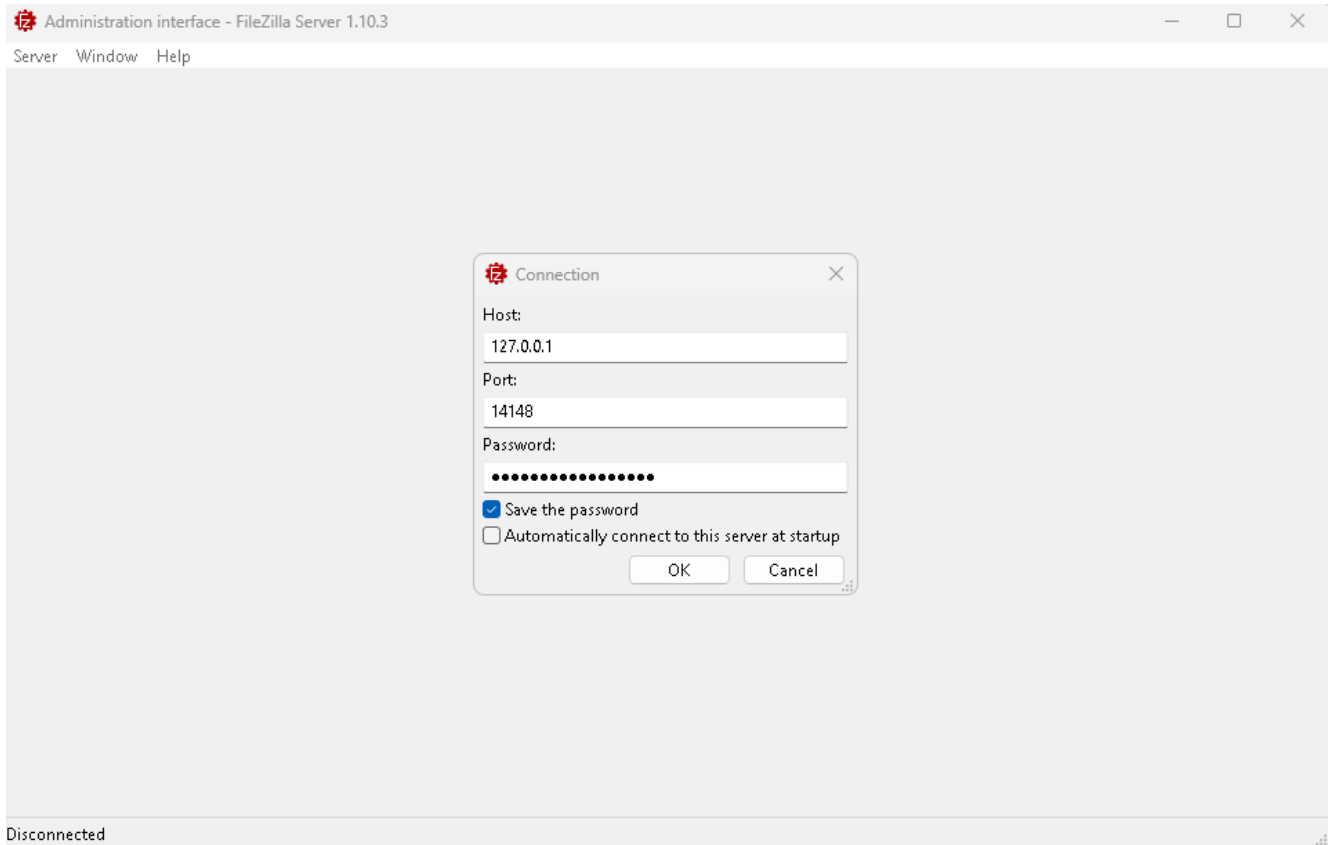
1. Cài đặt FileZilla Server

- Bước 1:** Trên máy ảo, vào trang chủ của [FileZilla](https://filezilla-project.org), bấm [Download FileZilla Server](#) và chọn bản phù hợp với hệ điều hành và phiên bản hệ điều hành của bạn (**32-bit hoặc 64-bit**), sau đó chọn tải phiên bản [FileZilla Server](#) miễn phí.



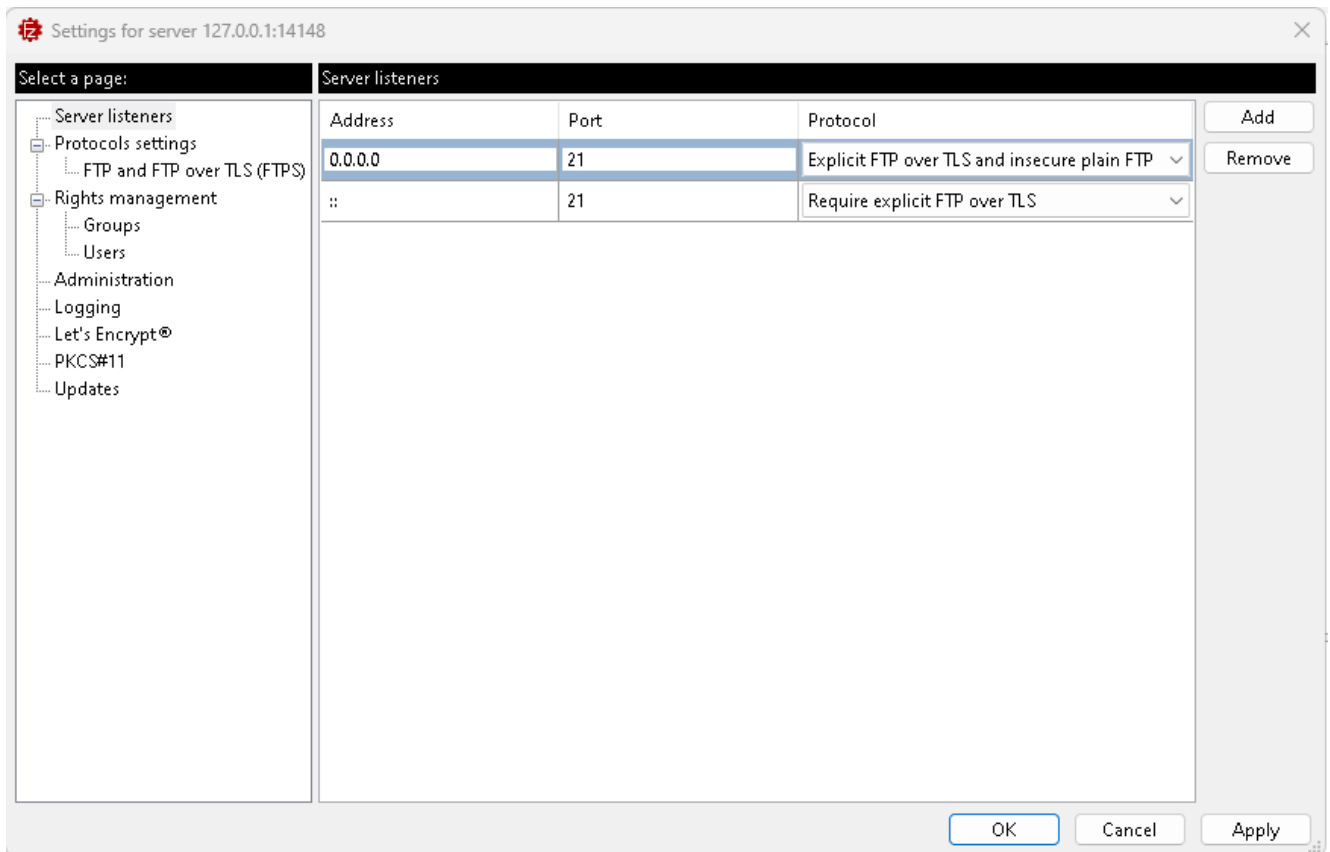
Nút Download FileZilla Server

- Bước 2:** Sau khi cài đặt file setup, chúng ta bấm nút [connect server](#) và nhập mật khẩu chúng ta đã lựa chọn ở bước setup hoặc có thể không nhập nếu không đặt mật khẩu.



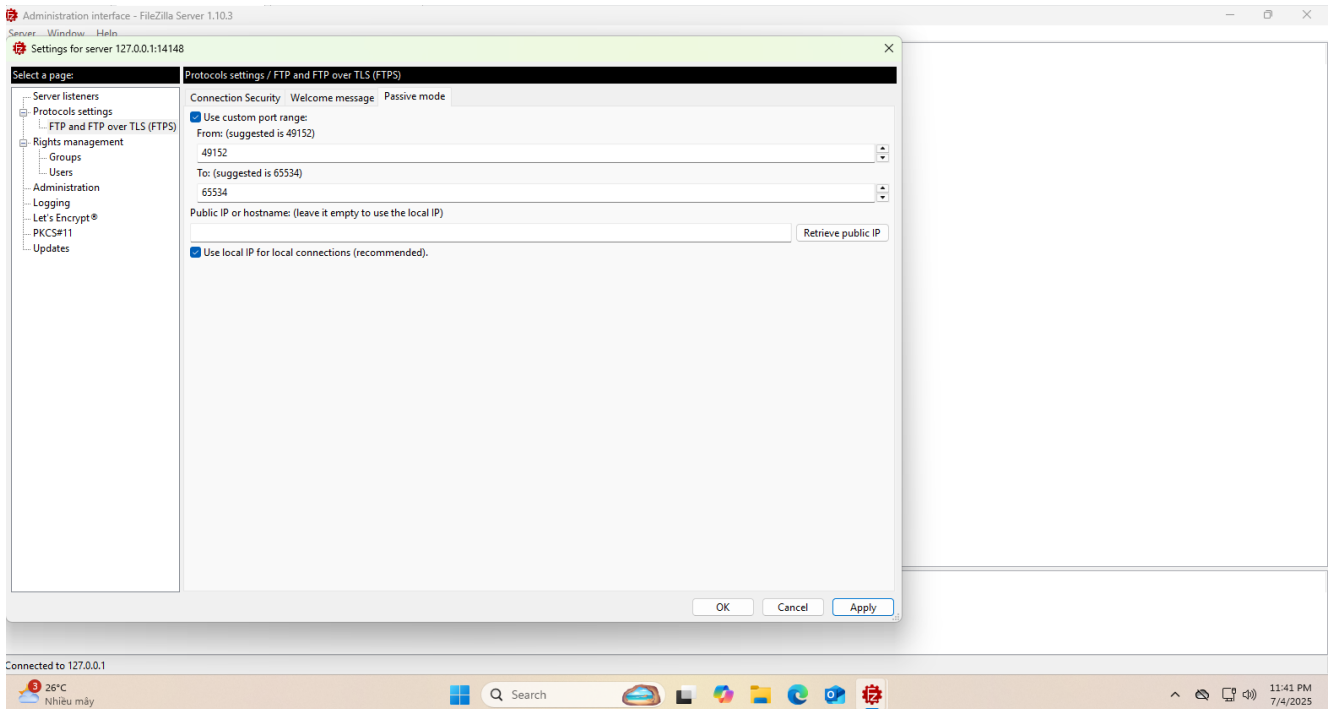
Đăng nhập vào FileZilla Server

- **Bước 3:** Vào góc trên cùng bên trái, chọn **Server** và bấm vào **Configure** để cấu hình server. Ở **Server listeners**, chọn **Explicit FTP over TLS and insecure plain FTP**.



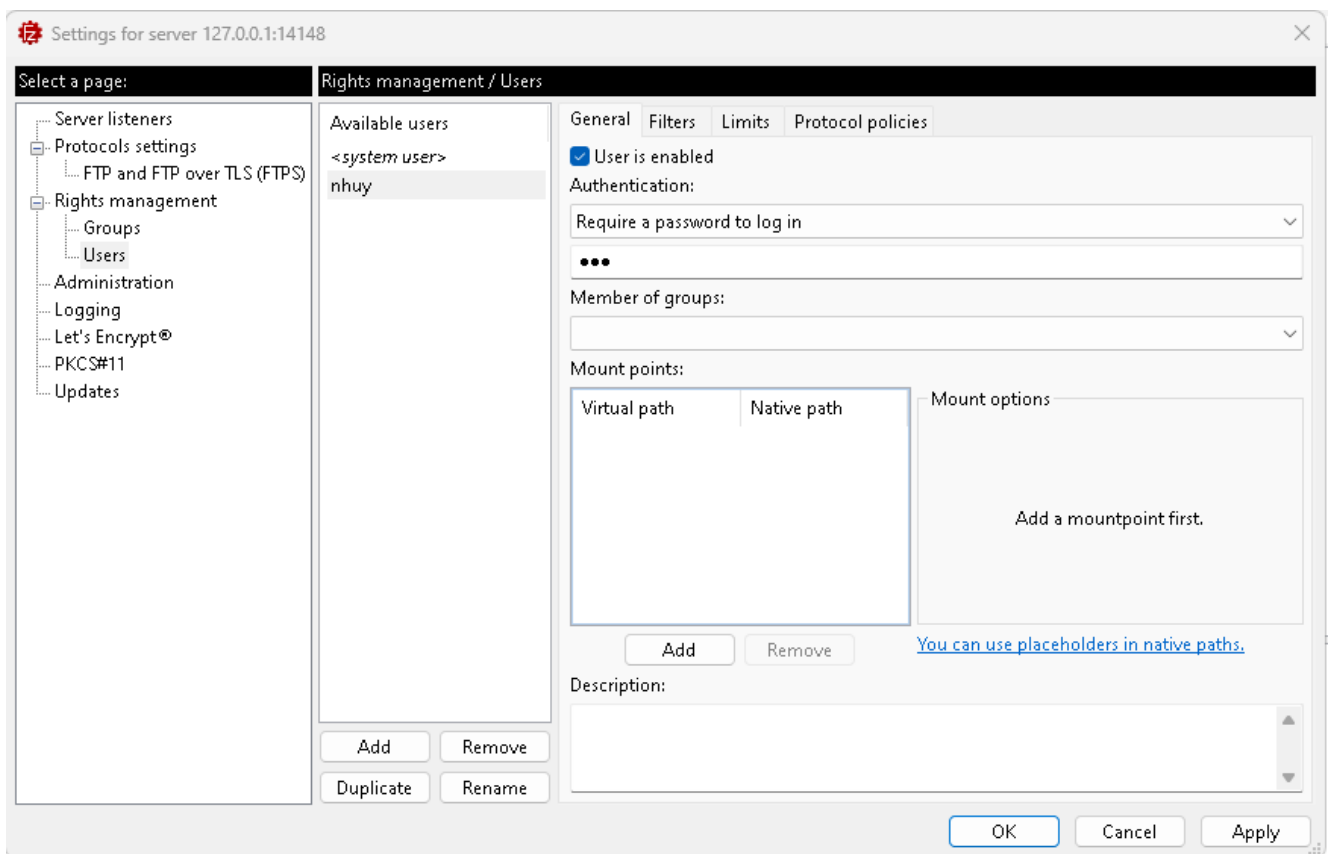
Cấu hình listener cho FileZilla Server

- **Bước 4:** Trong **FTP and FTP over TLS (FTPS)** của **Protocols settings**, vào **Passive mode** tick vào **Use custom port range** để server có thể sử dụng các cổng trong khoảng này, mặc định là **49152-65534** và có thể thay đổi nếu muốn, để gửi nhận dữ liệu trong chế độ kết nối Passive.



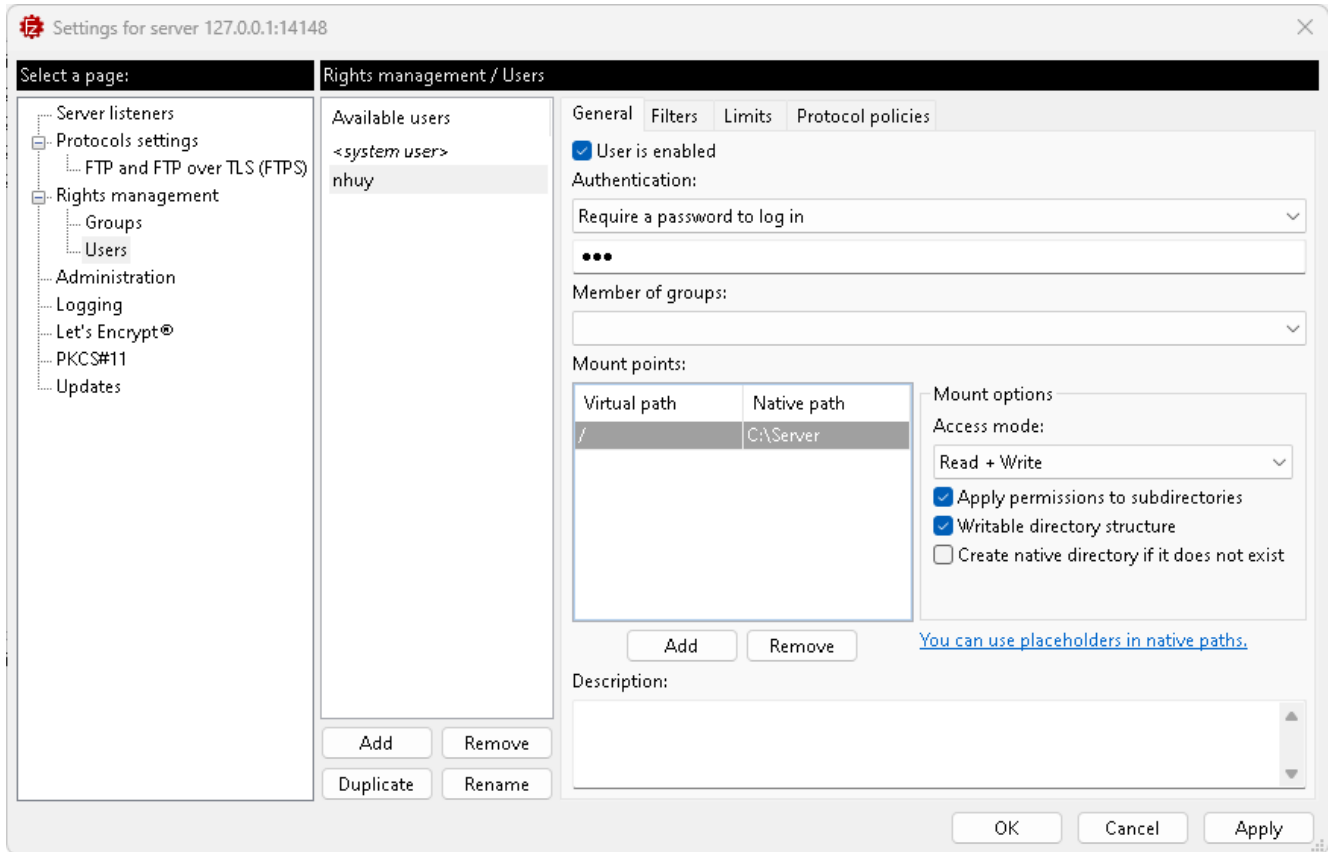
Bật và custom các cổng dùng trong chế độ Passive

- **Bước 5:** Trong **Rights management**, vào **Users** và bật **User is enabled**, tiếp theo nhấn **Add** ở **Available users** và nhập tên người dùng, ở mục **Authentication** chọn **Require a password to log in** và tiến hành nhập mật khẩu, dùng để xác thực khi người dùng muốn mở kết nối đến **server**.



Thêm người dùng vào FileZilla Server

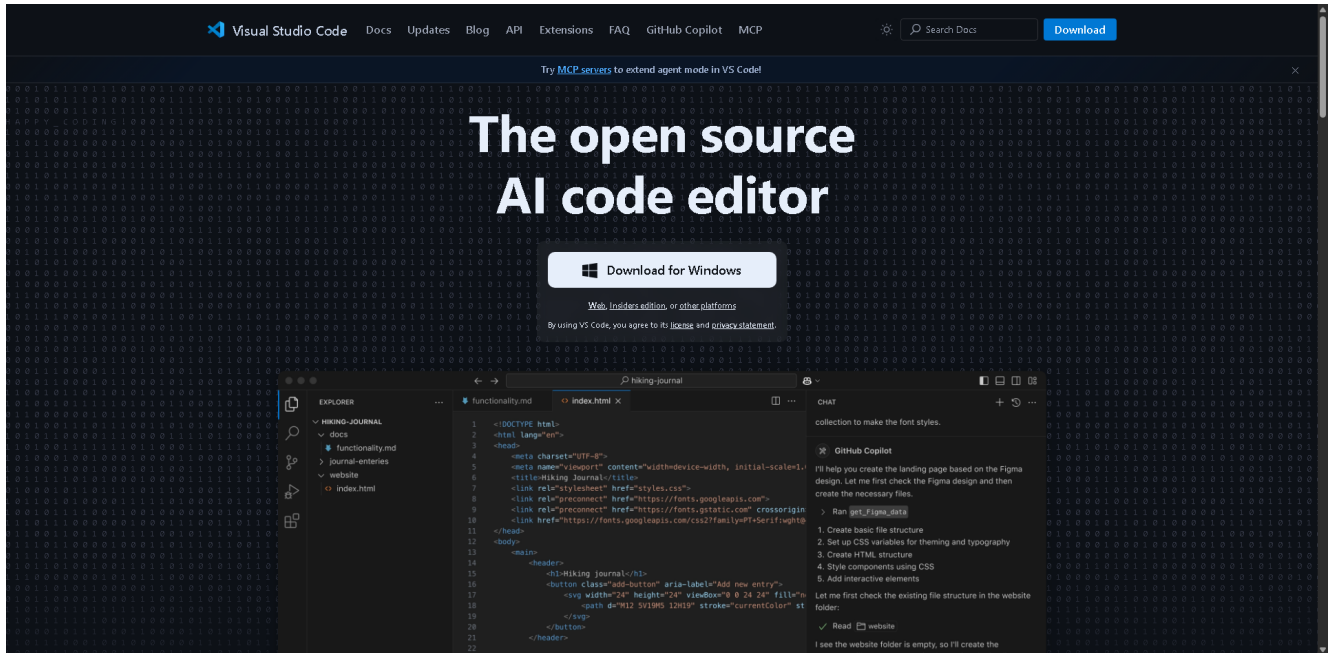
- **Bước 6:** Trong **Mount points**, nhấn **Add** để thêm đường dẫn đến thư mục chứa tệp tin khi được gửi từ **client**. Giữ nguyên **Virtual Path** là **/** và nhập vào **Native Path** đường dẫn tuyệt đối đến thư mục chứa file trên **server** mà bạn muốn người dùng thao tác, để ở chế độ **Read + Write**, nếu **Native Path** là đường dẫn đến thư mục chưa tồn tại thì bạn có thể tick thêm **Create native directory if it does not exist**. Cuối cùng nhấn **Apply** và cấu hình **FileZilla Server** đã hoàn tất.



Cấu hình đường dẫn đến thư mục chứa file

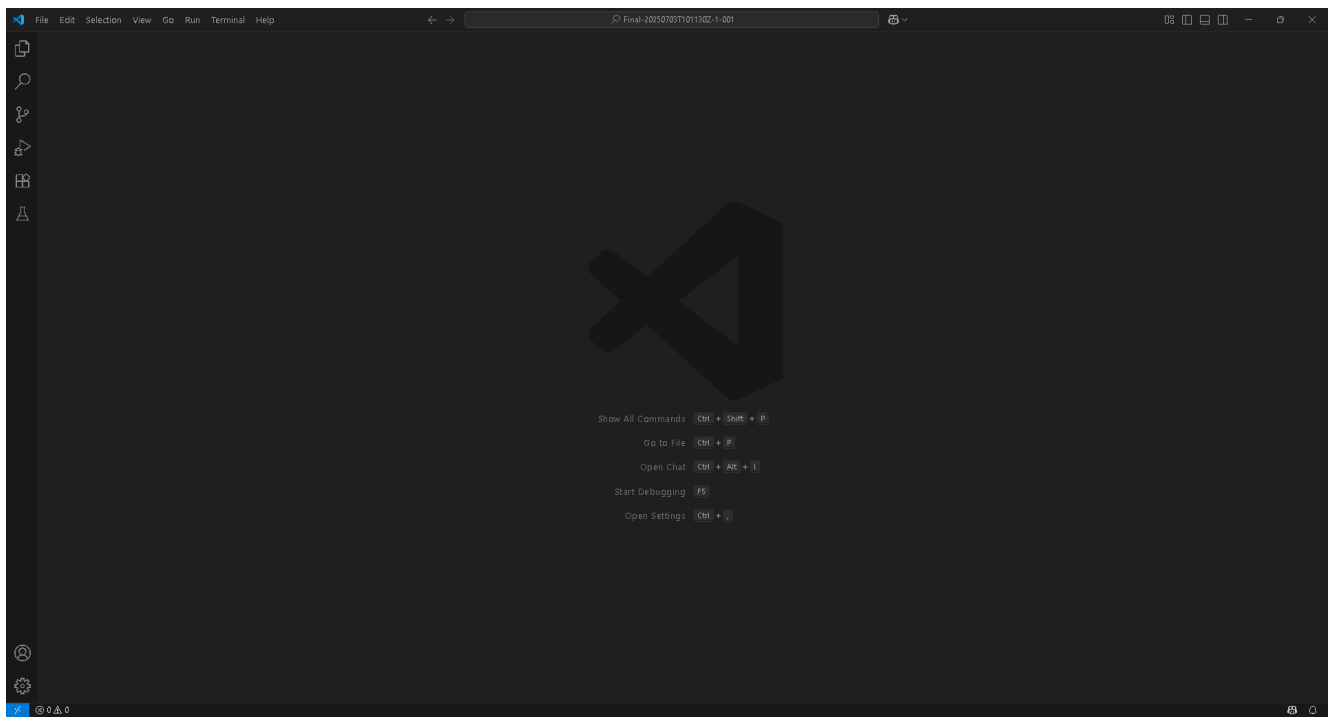
2. Cài đặt Visual Studio Code (VSCode) và Python

- **Bước 1:** Trên cả máy ảo và máy thực, vào trang chủ của **VSCode**, nhấn **Download for Windows** để tải xuống rồi cài đặt **Visual Studio Code**.



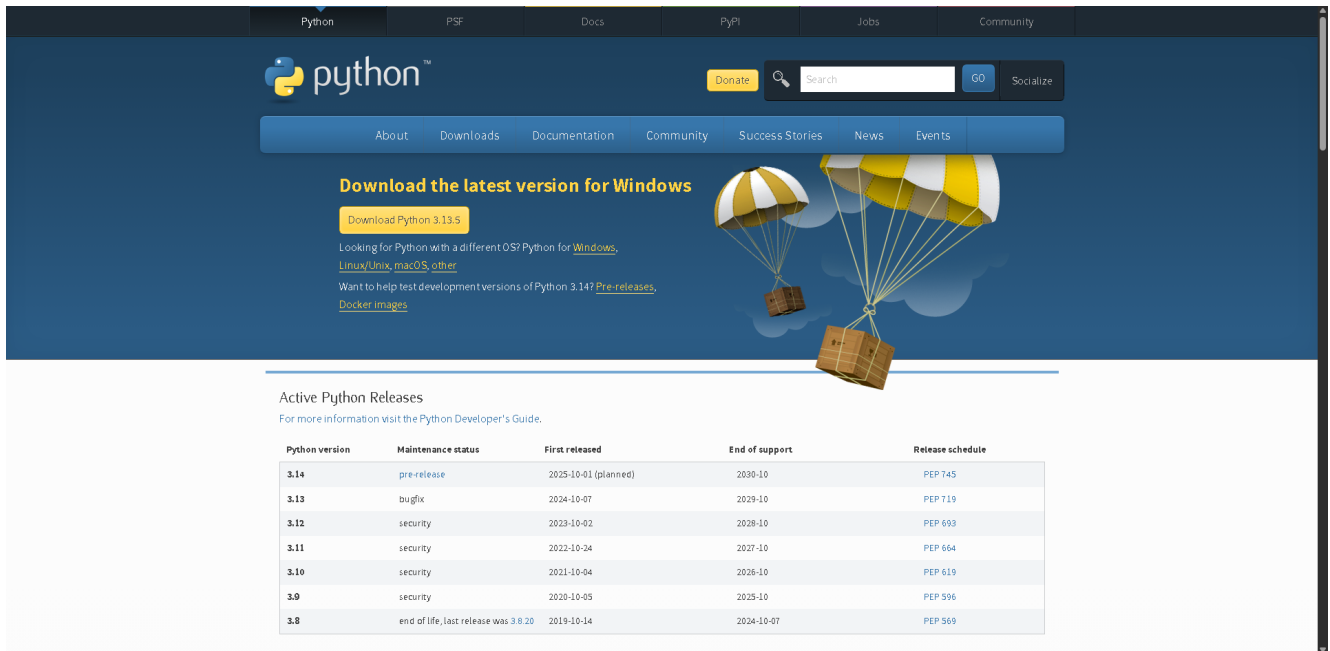
Download for Windows

- **Bước 2:** Mở file setup, chấp nhận điều khoản, chọn thư mục cài đặt và nhấn **Install**. Sau khi cài đặt xong, chọn **Launch Visual Studio Code** và nhấn **Finish**. Cửa sổ VSCode sẽ hiện ra như sau:



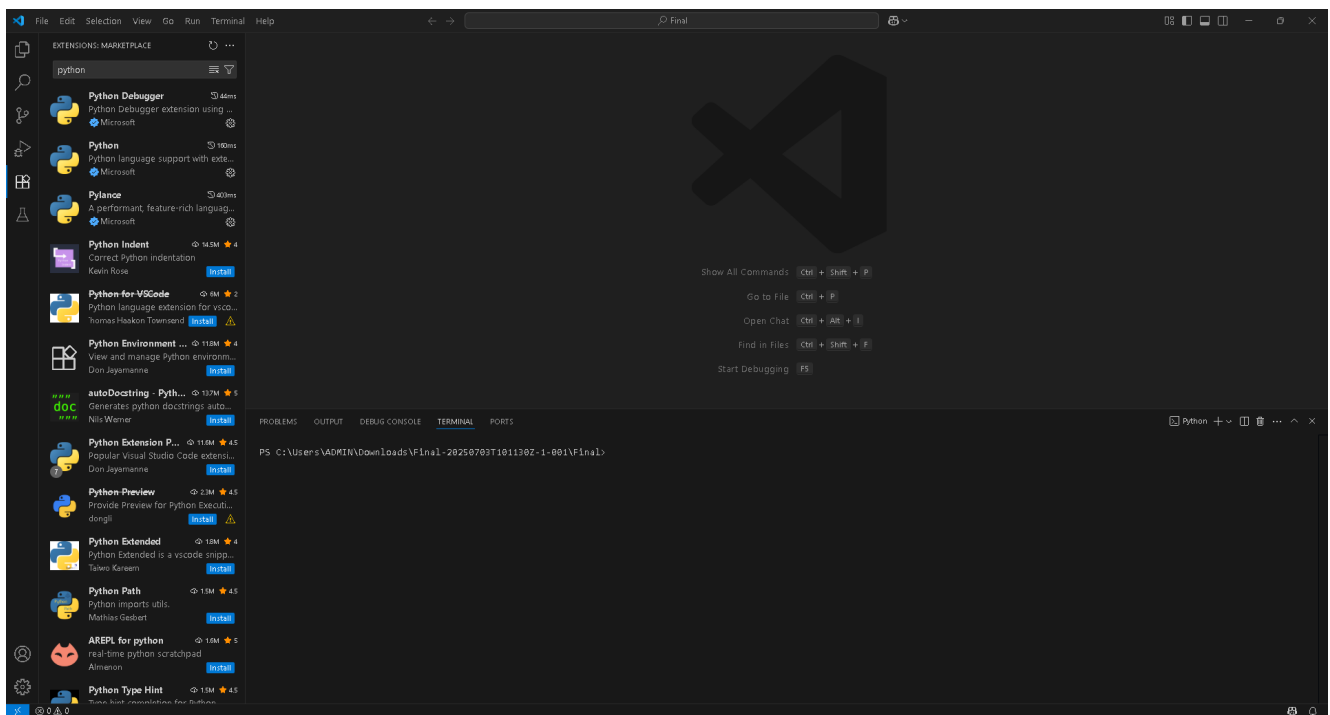
Cửa sổ VSCode sau khi cài đặt

- **Bước 3:** Trên cả máy ảo và máy thực, vào trang chủ của **Python**, tải xuống phiên bản Python mới nhất. Chạy file setup, giữ nguyên cài đặt mặc định (nhớ tick vào **Add python.exe to PATH** đối với các máy cài python lần đầu) và nhấn **Install**, bạn đã thành công cài đặt Python.



Cài đặt Python

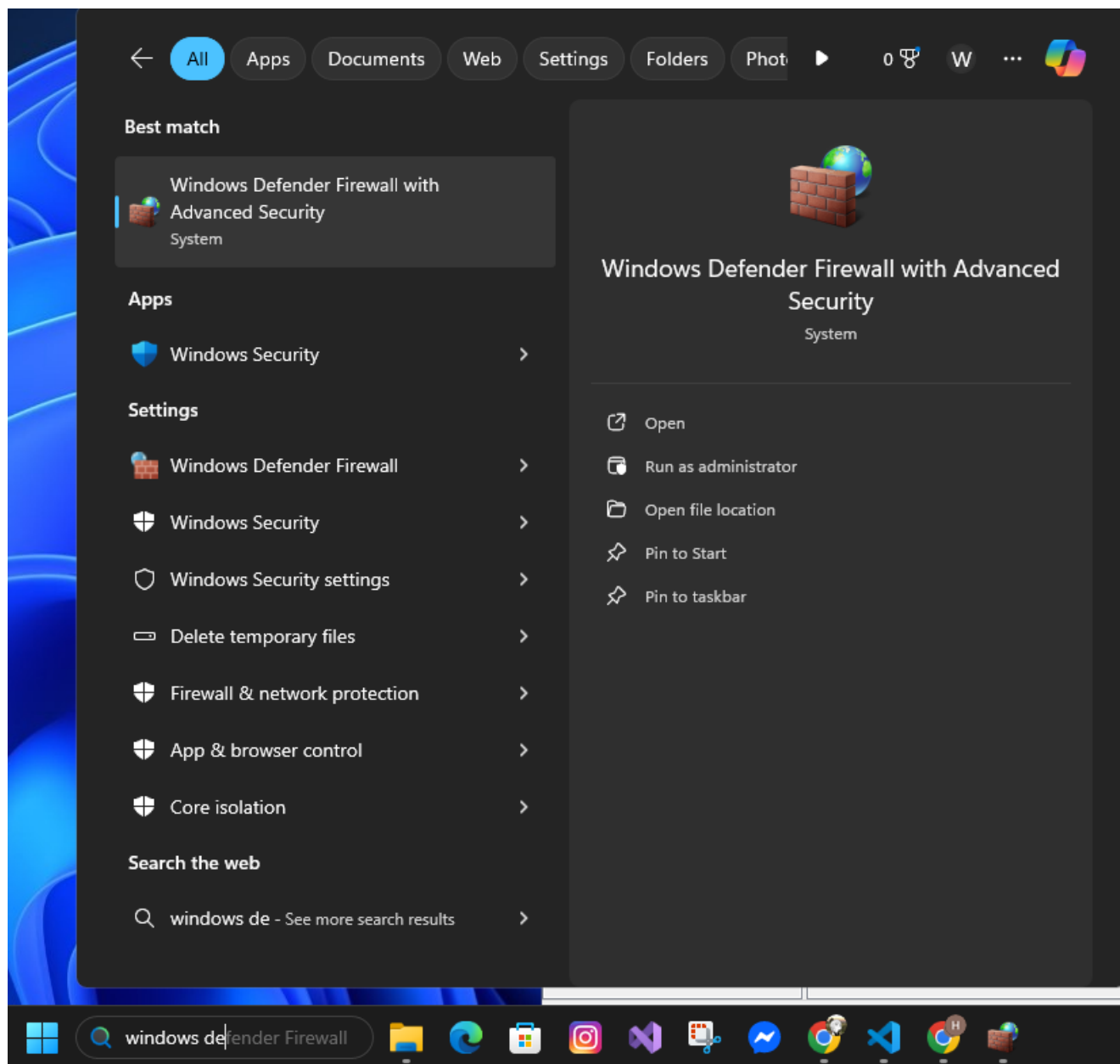
- Bước 4:** Tại cửa sổ **VSCode**, ở thanh dọc góc trên cùng bên trái nhấn chọn **Extension** rồi tải **Python Debugger** và **Python**.



Tải extension để lập trình Python

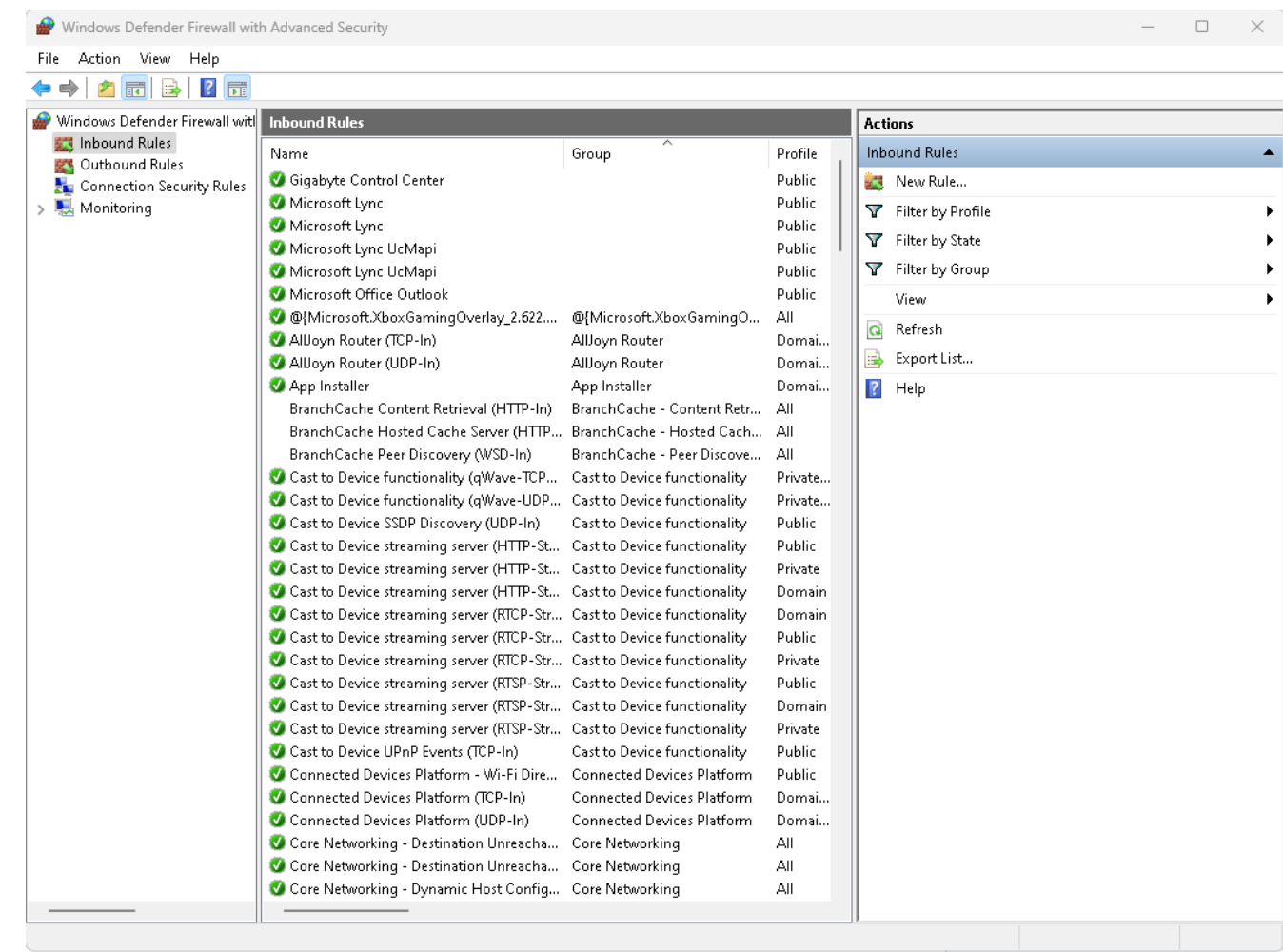
3. Cấu hình Windows Defender Firewall (tường lửa)

- Bước 1:** Vào **Window Search Bar** - thanh tìm kiếm, nhập **Windows Defender Firewall with Advanced Security** và nhấp chuột vào.



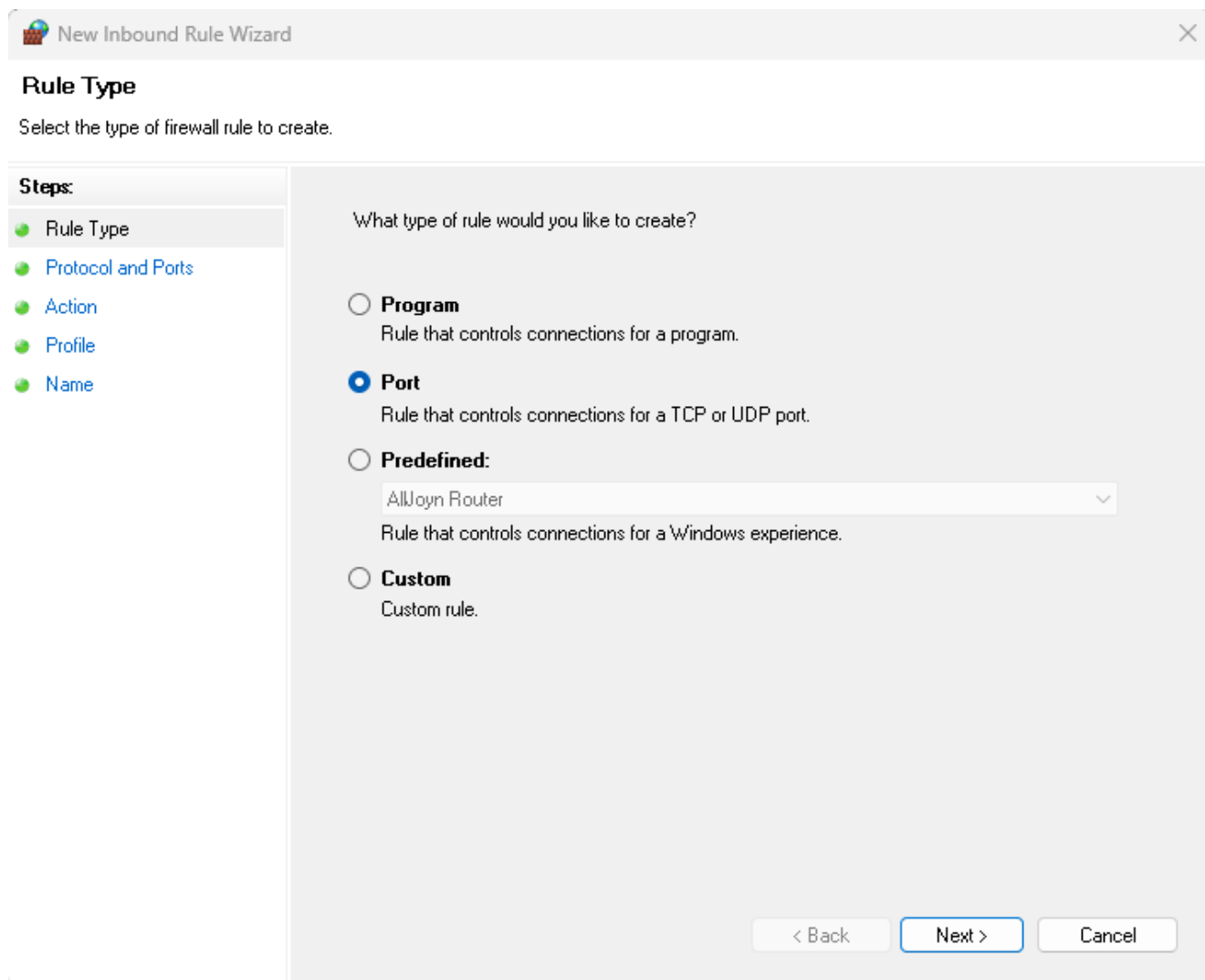
Windows Defender Firewall with Advanced Security

- **Bước 2:** Nhấp vào **Inbound Rules** ở bên trái và nhấp vào **New Rule** bên phải.



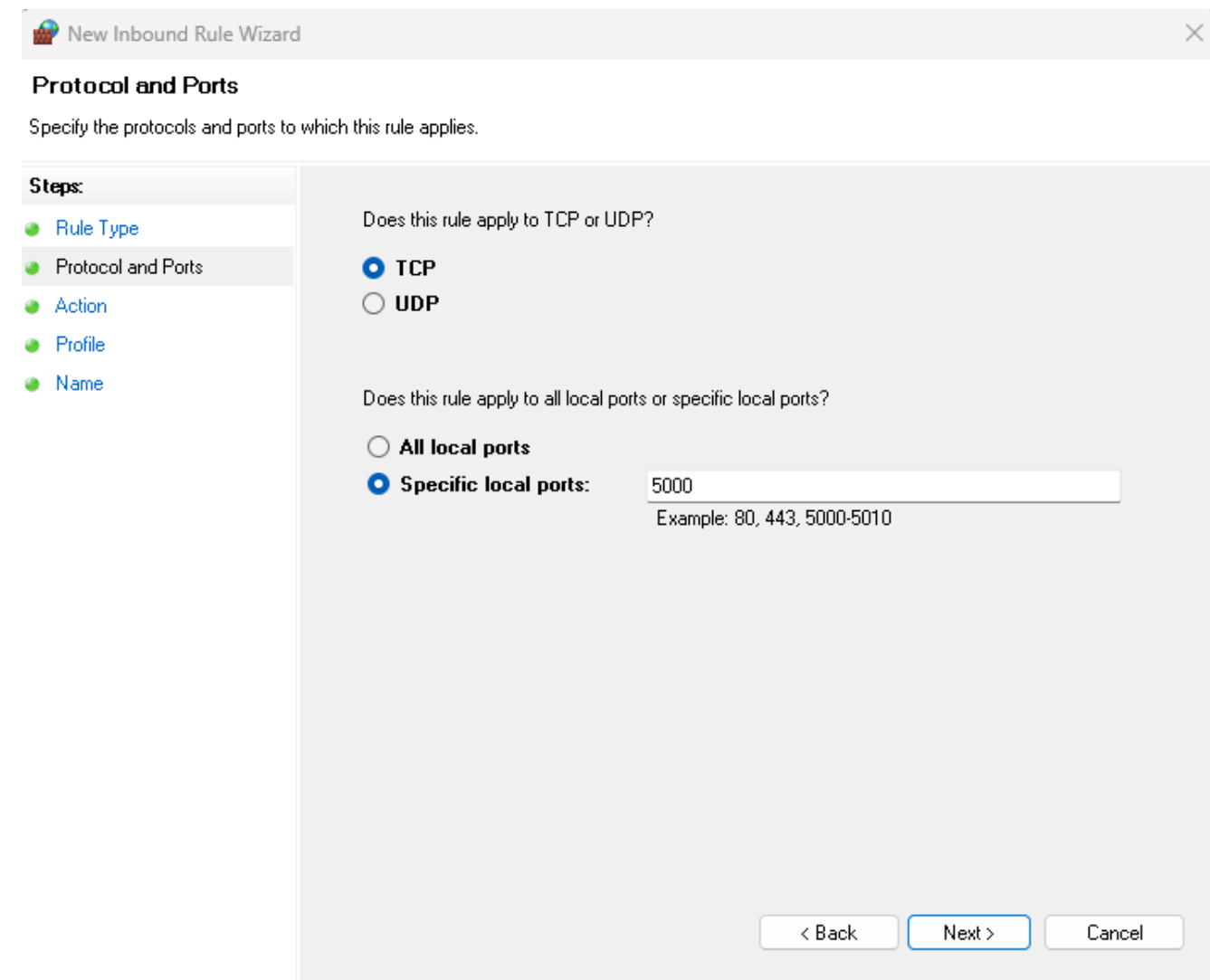
Nhấp lần lượt vào Inbound Rules và New Rule

- **Bước 3:** Chọn **Port** và nhấn **Next**.



Tạo Rule cho Port

- **Bước 4:** Chọn kết nối **TCP**, sau đó:
 - Ở máy ảo, nhập **21**, **5001**, **49152-65534** vào dòng **Specific local ports** để tường lửa không chặn những port có số hiệu này (*49152-65534 là từ port 49152 đến 65534 dùng trong chế độ kết nối Passive*), và nhấn **Next**.
 - Ở máy thực, nhập **5000** vào dòng **Specific local ports** để tường lửa không chặn port 5000 dùng trong chế độ kết nối Active, và nhấn **Next**.



New Inbound Rule Wizard

Protocol and Ports

Specify the protocols and ports to which this rule applies.

Steps:

- Rule Type
- Protocol and Ports**
- Action
- Profile
- Name

Does this rule apply to TCP or UDP?

☒ **TCP**

☐ **UDP**

Does this rule apply to all local ports or specific local ports?

☐ **All local ports**

☒ **Specific local ports:**

Example: 80, 443, 5000-5010

< Back Next > Cancel

Không chặn Port 5000 trên máy thực

- **Bước 5:** Chọn **Allow the connection** và nhấn **Next**.

New Inbound Rule Wizard

Action

Specify the action to be taken when a connection matches the conditions specified in the rule.

Steps:

- Rule Type
- Protocol and Ports
- Action**
- Profile
- Name

What action should be taken when a connection matches the specified conditions?

☒ **Allow the connection**
This includes connections that are protected with IPsec as well as those are not.

☐ **Allow the connection if it is secure**
This includes only connections that have been authenticated by using IPsec. Connections will be secured using the settings in IPsec properties and rules in the Connection Security Rule node.
[Customize...](#)

☐ **Block the connection**

< Back Next > Cancel

Allow the connection

- **Bước 6:** Tick chọn tất cả và bấm **Next**.

New Inbound Rule Wizard

Profile

Specify the profiles for which this rule applies.

Steps:

- Rule Type
- Protocol and Ports
- Action
- Profile**
- Name

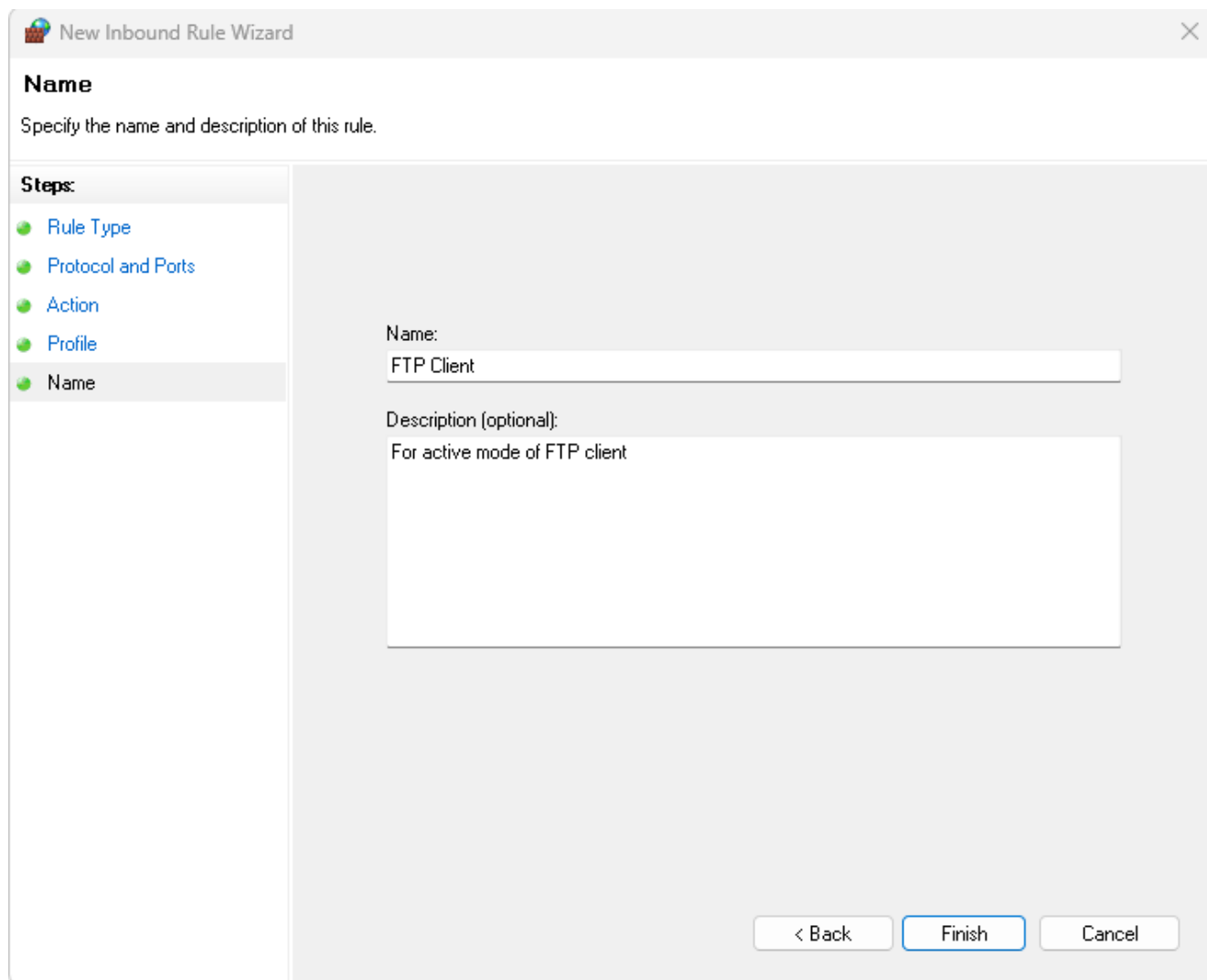
When does this rule apply?

- ☒ **Domain**
Applies when a computer is connected to its corporate domain.
- ☒ **Private**
Applies when a computer is connected to a private network location, such as a home or work place.
- ☒ **Public**
Applies when a computer is connected to a public network location.

< Back Next > Cancel

Tick chọn tất cả

- **Bước 7:** Đặt tên là **FTP Client** hoặc 1 tên nào đó dễ liên hệ, ở phần **Description** có thể ghi: **For active mode of FTP client**, với máy ảo có thể sửa lại những thông tin này cho phù hợp, và cuối cùng chọn **Finish**.



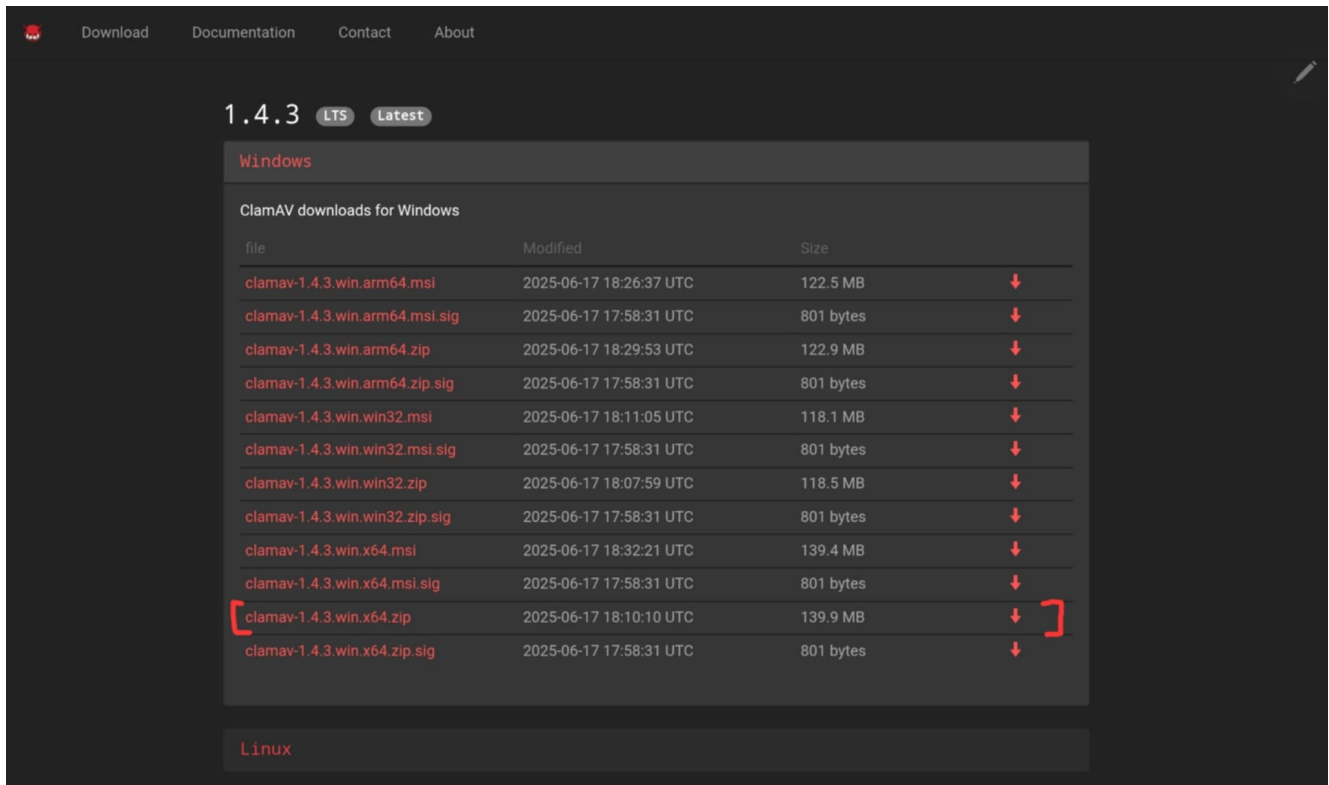
Đặt tên và mô tả cho Rule trên máy thực

II. Cài đặt và sử dụng ClamAV

1. Tải và chạy phần mềm ClamAV trên máy ảo

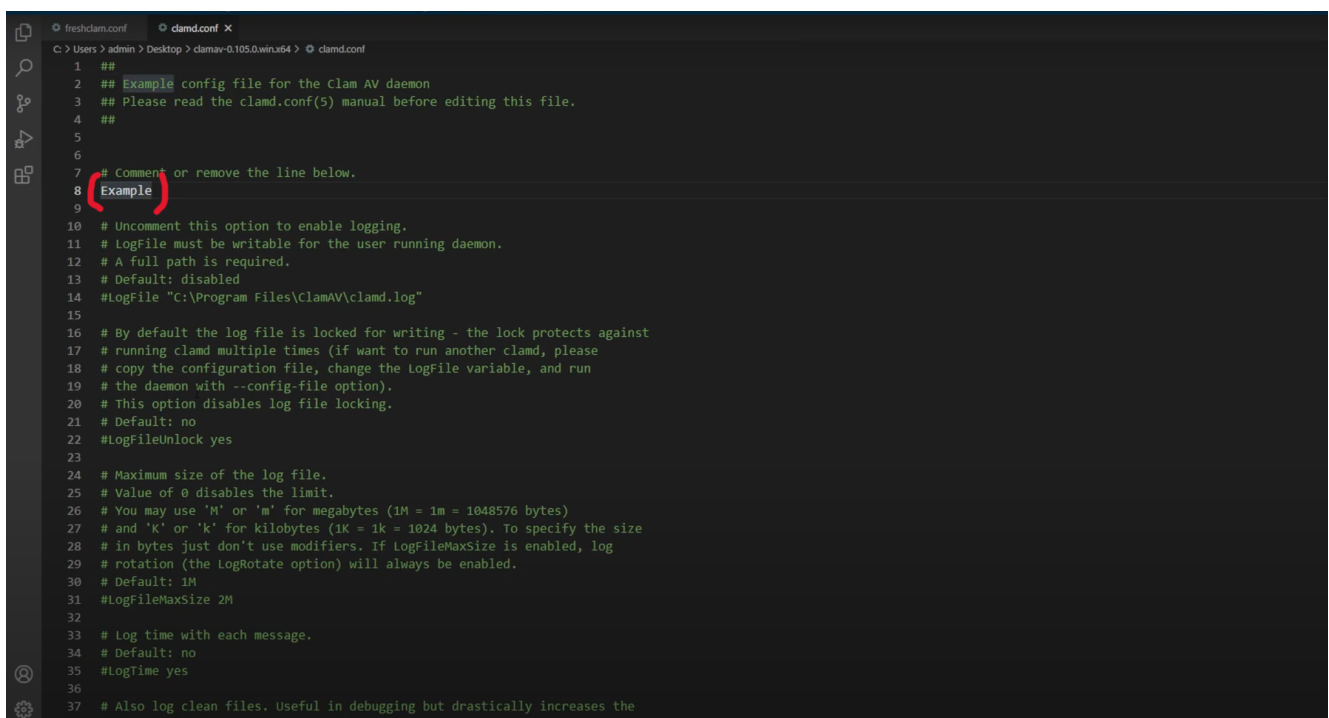
Bước 1: Tải ClamAV và set up ban đầu

- Vào trình duyệt trên máy ảo đang chạy FileZilla Server.
- Vào trang chủ của [ClamAV](#), đi đến phiên bản mới nhất hiện tại **1.4.3** và tải xuống file cài đặt phù hợp với hệ điều hành và phiên bản hệ điều hành của bạn (**32-bit hoặc 64-bit**).
- Đối với hệ điều hành Windows 11 bản 64-bit, bạn sẽ tải xuống file **clamav-1.4.3.win-x64.zip**



File cần tải

- Vào thư mục chứa file **.zip** đã tải về, giải nén và mở folder **clamav-1.4.3.win.x64** (hoặc tên file tương ứng với phiên bản bạn đã tải về).
- Vào folder **conf_examples**, mang 2 file **clamd.conf.sample** và **freshclam.conf.sample** ra folder chính (**clamav-1.4.3.win-x64.zip**).
- Tiến hành đổi tên (xóa đuôi **.sample**) 2 file kể trên lần lượt thành **clamd.conf** và **freshclam.conf**. Sau đó, mở 2 file này bằng trình soạn thảo (VS Code, Notepad, ...), tìm dòng **Example**, xóa dòng này đi và lưu 2 file lại.



Dòng example cần xóa trong cả 2 file

Bước 2: Cấu hình ClamAV

- Mở **Command Prompt (CMD)** và nhập lệnh `cd <đường dẫn tuyệt đối đến folder clamav-1.4.3.win-x64>`. Ví dụ:

```
cd C:\Users\duynh\Downloads\clamav-1.4.3.win.x64\clamav-1.4.3.win.x64
```

--> Nhờ đó, bạn sẽ dễ dàng truy cập đến các file thuộc folder này mà không cần phải nhập lại đường dẫn tuyệt đối, chỉ cần nhập tên file là được.

- Tiếp theo, nhập `freshclam` hay `freshclam.exe` để cập nhật cơ sở dữ liệu virus của ClamAV. Quá trình cập nhật sẽ diễn ra và khi hoàn tất, bạn có thể sử dụng ClamAV để quét các file. --> Nếu không chạy `freshclam` trước, ClamAV vẫn có thể hoạt động nhưng sẽ quét các file rất lâu và không chính xác do chưa cập nhật cơ sở dữ liệu virus mới nhất.

--> Ngược lại, khi đã chạy `freshclam`, ClamAV sẽ quét các file một cách nhanh chóng và tính tin cậy của các file khi được gửi lên server sẽ được đảm bảo.

Bước 3: Quét thử file bằng ClamAV

- Sau khi cập nhật xong, bạn đã có thể quét file bằng cách nhập `clamscan <đường dẫn tuyệt đối đến file cần quét>` (*clamscan trong lệnh trên có thể thay thế bằng `clamscan.exe`*). Ví dụ:

```
clamscan C:\dnhat\city.txt
```

--> Nếu file không có virus, bạn sẽ nhận được thông báo như sau:

```
C:\dnhat\city.txt: OK
```

2. Vận hành ClamAV thông qua Python trên máy ảo

Ở phần chạy thử bên trên, bạn có thể thấy rằng việc quét file bằng ClamAV thông qua **Command Prompt** khá bất tiện, chỉ quét được các file trên máy ảo hiện hành và phải nhập lệnh thủ công. Do đó, chúng ta sử dụng Python để phục vụ việc quét file từ máy **client** trước khi gửi lên **server** (*FileZilla Server*) một cách thuận tiện hơn.

- Thư viện cần sử dụng:

```
import os
import subprocess
import socket
import msvcrt
```

--> Thư viện **os** dùng để quản lý tạm thời các file được gửi đến từ server.

--> Thư viện **subprocess** dùng để chạy các lệnh của ClamAV từ Python.

--> Thư viện **socket** dùng để tạo *socket* phục vụ việc lắng nghe, nhận file và gửi kết quả quét về máy **client**.

--> Thư viện **msvcrt** dùng để nhận input từ bàn phím, phục vụ việc dừng chương trình **agent**.

- Tạo các biến lưu thông tin của Agent:

```
HOST = '0.0.0.0'    #Địa chỉ IP mà Agent lắng nghe
PORT = 5001         #Cổng lắng nghe
BUFFER_SIZE = 8192  #Kích thước bộ đệm để file
SAVE_PATH           #Địa chỉ lưu file tạm thời
ClamAV_PATH         #Địa chỉ tuyệt đối đến file clamscan.exe
```

--> Địa chỉ IP **0.0.0.0** ở HOST có nghĩa là lắng nghe tất cả các địa chỉ IP từ Agent trên máy ảo, khiến cho việc các máy client khác nhau có thể gửi file cho Agent kiểm tra một cách linh hoạt.

--> Cổng **5001** là cổng mà Agent lắng nghe các lệnh kết nối từ máy client. Bạn có thay đổi cổng này nếu muốn, nhưng phải đảm bảo cổng không bị chiếm dụng bởi ứng dụng khác và không bị tường lửa chặn.

--> **BUFFER_SIZE** là **8192** bytes, là kích thước bộ đệm phổ biến để đọc hoặc ghi file. Kích thước này phù hợp với có thể đọc nhanh các file lớn mà không bị tràn bộ nhớ.

- Phương thức **makedirs()** từ thư viện **os** sẽ nhận vào **SAVE_PATH** và tạo ra file tạm thời để lưu trữ nội dung của file được gửi từ máy client. Nếu file từ địa chỉ **SAVE_PATH** thì lệnh sẽ không làm gì cả.

```
os.makedirs(SAVE_PATH, exist_ok=True)
```

- Lần lượt dùng các phương thức **socket()**, **bind()**, **listen()** để khởi tạo socket cho **server** và dùng socket này để lắng nghe yêu cầu từ kết nối từ **client**. Ngoài ra, phương thức **settimeout()** cũng được sử dụng để đặt thời gian chờ phản hồi từ **client** là **5.0** giây, phục vụ việc xử lý dấu cách ' ' là input nhập từ bàn phím để dừng chương trình của **agent**.

```
server_socket = socket.socket()
server_socket.bind((HOST, PORT))
server_socket.listen()
server_socket.settimeout(5.0)
```

- Khi đã vào vòng lặp hoạt động, **agent** luôn lắng nghe yêu cầu kết nối từ **client** kể cả sau khi đã thực hiện quét xong file và đóng socket nhận dữ liệu. Nếu người dùng nhập dấu cách ' ' từ bàn phím, vòng lặp sẽ thoát ra, socket lắng nghe của **agent** sẽ đóng lại và chương trình sẽ ngừng. Mã nguồn dưới đây sẽ thực hiện việc nhận input dấu cách ' ' từ bàn phím.

```
if msvcrt.kbhit():
    key = msvcrt.getch()
    if key == b' ':
        print("Đã nhận lệnh dừng server.")
        break
```

- Dùng phương thức `accept()` để chấp nhận kết nối từ yêu cầu kết nối từ client. Đồng thời, phương thức này trả về một tuple gồm socket mới được mở để truyền gửi thông tin cho **client** và thông tin địa chỉ của **client**.

```
conn, addr = server_socket.accept()
```

- Dùng phương thức `recv()` để nhận dữ liệu từ **client**. Dữ liệu này sẽ được lưu vào biến `data` và được ghi vào file tạm theo địa chỉ `SAVE_PATH` nếu không rỗng.

```
with open(SAVE_PATH, 'wb') as f:
    while True:
        data = conn.recv(BUFFER_SIZE)
        if not data:
            break
        f.write(data)
```

- Sau khi đã hoàn thành việc ghi vào file tạm, phương thức `subprocess.run()` sẽ được gọi, nhận vào lần lượt `ClamAV_PATH` và `SAVE_PATH`, để quét file này bằng ClamAV. Kết quả sẽ được lưu vào biến `result` sau đó được trả về cho **client** thông qua phương thức `send()`.

```
result = subprocess.run(
    [ClamAV_PATH, SAVE_PATH],
    capture_output=True,
    text=True, timeout = 120
)

conn.send(result.stdout.encode())
```

- Cuối cùng, đóng kết nối giữa **client** và **server** sau khi đã hoàn thành việc quét file và gửi kết quả về.

```
conn.close()
```

III. Hướng dẫn chạy chương trình

1. Tổ chức mã nguồn của client

- Mã nguồn của **client** được chia ra thành các file **.py** khác nhau để dễ quản lý và bảo trì. Các file **.py** phục vụ các chức năng khác nhau của **client**.



Mã nguồn được chia thành các file **.py**

- Các file mã nguồn của **client** bao gồm:
 - ftp_core.py**: Chứa các hàm thực hiện các chức năng hỗ trợ của **client** như gửi lệnh và nhận phản hồi từ **server**.
 - globals.py**: Chứa các biến toàn cục của **client** như socket gửi lệnh đến server và các thông tin về trạng thái của **client**.
 - operations.py**: Chứa các hàm thực hiện các chức năng của **client** như liệt kê file, xóa file, đổi tên file,...Các hàm này sẽ được gọi lại trong mã nguồn thuộc file **utils.py**.
 - session.py**: Chứa các hàm thực hiện các chức năng liên quan đến phiên làm việc của **client** như kết nối, ngắt kết nối, in ra trạng thái,...
 - updown.py**: Chứa các hàm thực hiện các chức năng gửi file và tải file xuống từ **server**.
 - utils.py**: Chứa các hàm gọi lại các hàm trong **operation.py** để bắt đầu thực hiện các chức năng liệt kê file, xóa file, đổi tên file,...
 - main.py**: Chứa hàm **main()** để chạy toàn bộ chương trình **client**. Để vận hành **client** chỉ cần chạy file này.
- Folder **__pycache__** là folder chứa file lưu cấu hình của client, khi biên dịch lần đầu sẽ được tạo ra để các lần biên dịch sau được nhanh hơn. Bạn không cần quan tâm đến folder này.

2. Vận hành ứng dụng

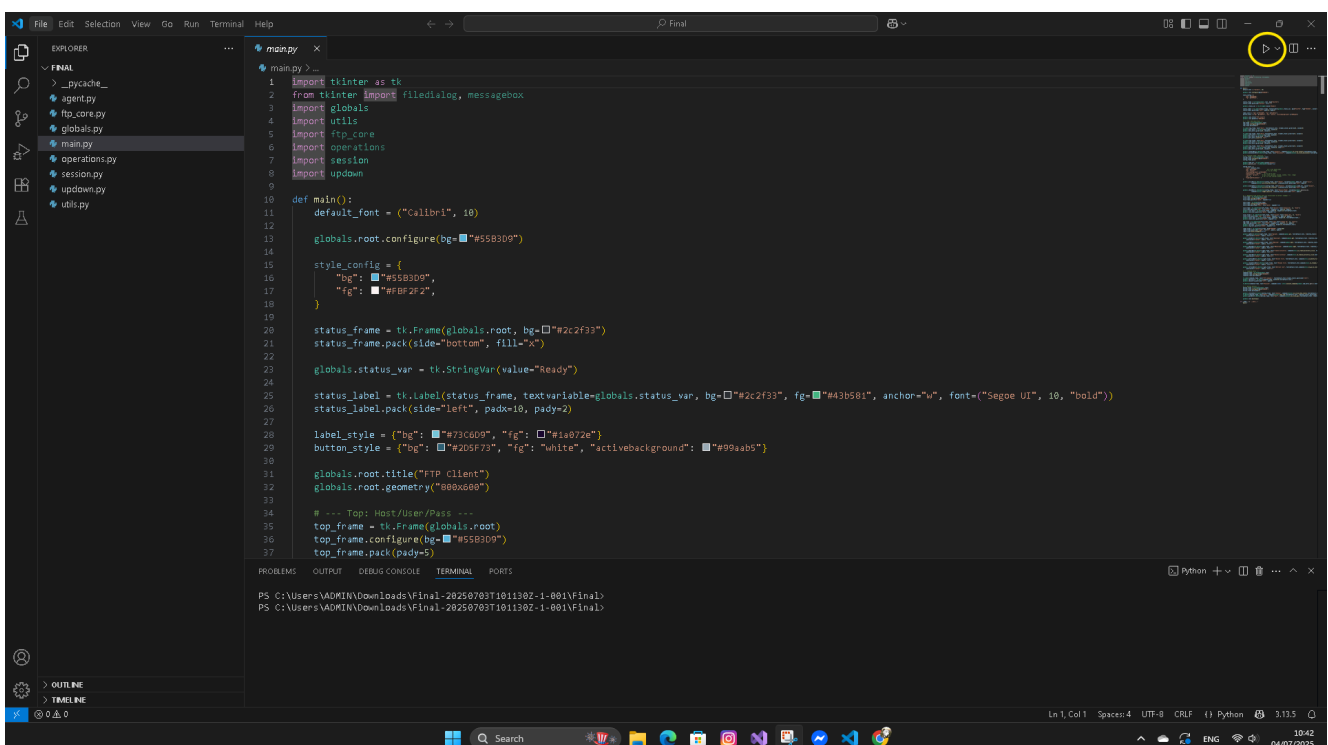
- Bước 1**: Trên máy thực, tải file **ftp_client.zip** về và tiến hành giải nén. Bạn sẽ nhận được folder **ftp_client** chứa mã nguồn của **client**. Trên máy ảo, tải file **clamav_agent.py** và tạo folder **agent**, có thể đặt tên khác, để chứa file này.
- Bước 2**: Sau khi tải thành công, bạn vào **File** và nhấn **Open Folder** để mở thư mục các thư mục trên. Chọn file **main.py** trong folder **ftp_client** và file **clamav_agent.py** trong folder **agent**. Lưu ý, bạn cần phải chỉnh sửa các thông tin sau trước khi chạy chương trình:

- Trong file **updown.py**, bạn cần chỉnh sửa lại địa IP **ClamAV_HOST** để phù hợp với máy đang chạy **agent** hiện tại.
 - Trong file **clamav_agent.py**, bạn cần chỉnh sửa lại đường dẫn tuyệt đối đến file **clamscan.exe** bạn đã tải về trên máy ảo trước đó.
- **Bước 4:** Trên máy ảo, nhấp vào shortcut **Administer FileZilla Server** hoặc vào thành tìm kiếm và gõ **Administer FileZilla Server** sau đó nhấp biểu tượng để chạy **FileZilla Server**.



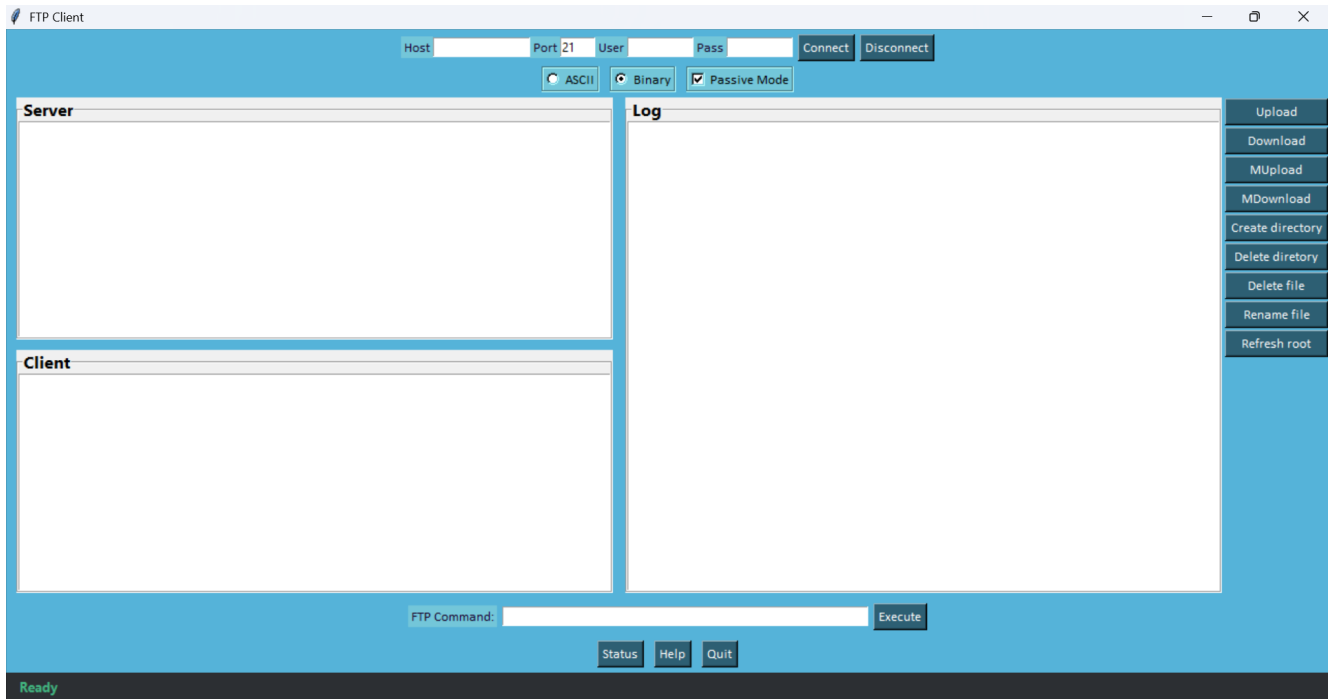
Administer FileZilla Server

- **Bước 5:** Trên máy thực, chọn file **main.py** và nhấn nút tam giác **Run** ở góc trên cùng bên phải để chạy chương trình của **client**. Trên máy ảo, chọn file **clamav_agent.py** và nhấn nút tương tự để chạy chương trình của **agent**.



Nút chạy chương trình

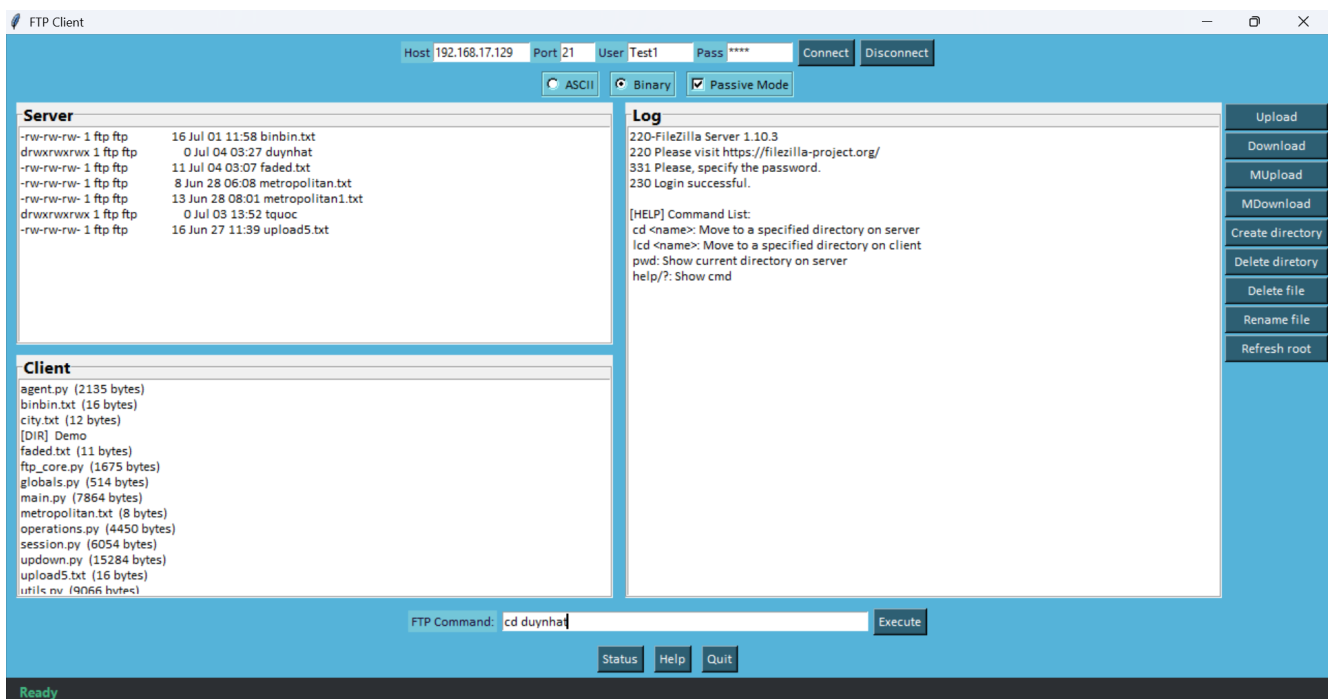
Như vậy, bạn đã hoàn thành việc chạy hệ thống. Giờ đây, bạn có thể kết nối, sử dụng các chức năng của **client** để giao tiếp và gửi nhận dữ liệu với **server** thông qua việc thao tác với giao diện.



Giao diện của client

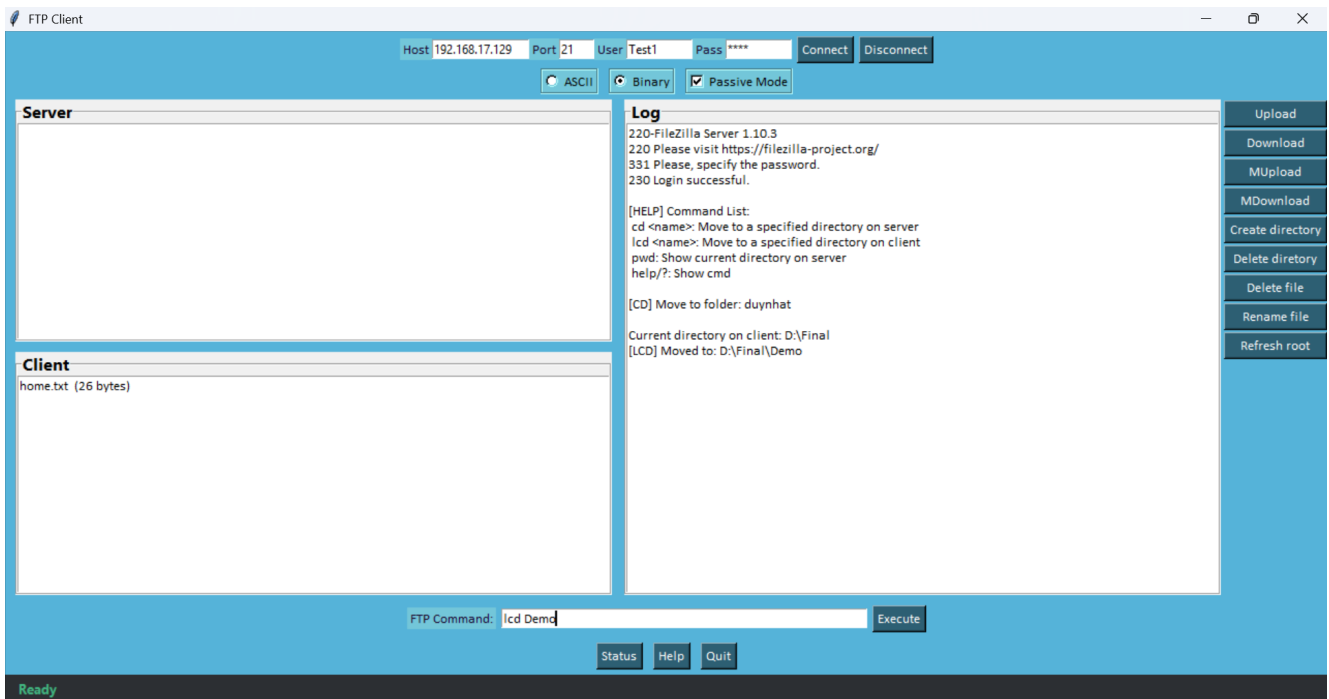
IV. Các lệnh mẫu

- **cd foldername**: Dẫn đến thư mục người dùng muốn trên **server** và cập nhật lại cửa sổ Server trên giao diện.



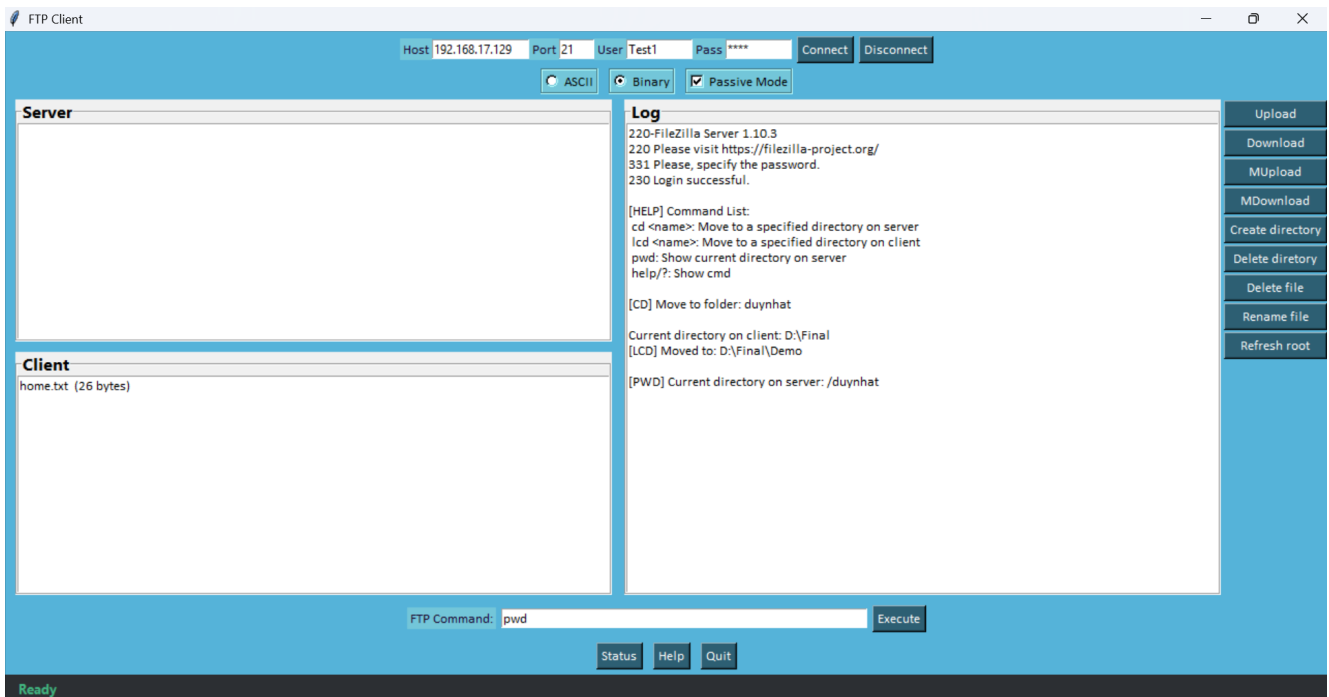
Lệnh cd hỗ trợ chuyển thư mục trên server

- **lcd foldername:** Dẫn đến thư mục người dùng muốn trên **client** và cập nhật lại cửa sổ Client trên giao diện.



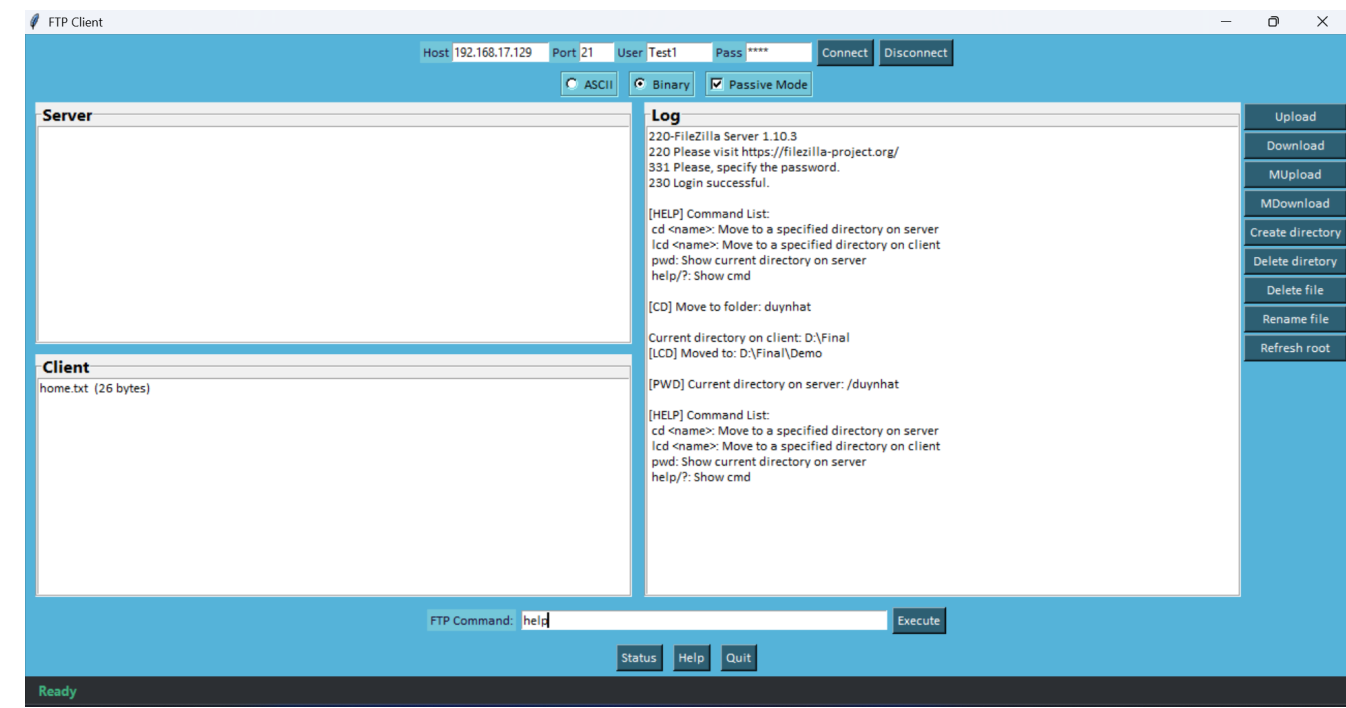
Lệnh lcd hỗ trợ chuyển thư mục trên client

- **pwd:** Hiển thị tên thư mục hiện tại trên **server**.



duynhat là thư mục hiện tại

- **help:** Hiển thị các lệnh có thể thực hiện của chương trình lên cửa sổ Log trên giao diện.



Các lệnh hệ thống hỗ trợ được hiển thị